

OpenAI Agents SDK

실무 가이드 · 지식 점검

경량 벤더 네이티브 에이전트 SDK — 핵심 개념 · 패턴 · 코드 · 함정

LangGraph 경쟁 솔루션 · Agent + Runner 루프 아키텍처

OpenAI Agents SDK 기준 (2026년 6월) · 대상: 풀스택 / MLOps / DevOps 엔지니어

난이도: 실무 핵심 · 본문 한국어 + 코드/용어 영문

1. Agents SDK란 무엇이고 언제 쓰는가
2. 핵심 멘탈 모델: Agent + Runner 루프
3. Agent — instructions·tools·구조화 출력
4. Tools — function tool과 호스팅 도구
5. Runner — 실행 루프와 RunResult
6. Handoffs — 에이전트 간 위임
7. Guardrails — 입출력 검증과 tripwire
8. Sessions — 대화 이력 관리
9. Context — 의존성 주입
10. Streaming
11. Tracing — 기본 내장 관측성
12. 멀티 에이전트 패턴
13. Agents SDK vs LangGraph 비교
14. 자주 하는 실수 모음
15. API 빠른 레퍼런스

지식 점검 퀴즈 (객관식 20문항)

정답 및 해설

1 Agents SDK란 무엇이고 언제 쓰는가

OpenAI Agents SDK는 실험작 Swarm을 계승한 **경량·프로덕션 지향 에이전트 SDK**(Python/TypeScript). 설계 철학은 "배울 가치가 있는 만큼만 기능을, 빠르게 익힐 만큼만 원시 개념을" — 핵심 프리미티브가 **Agents · Tools · Handoffs · Guardrails · Sessions** 다섯 개뿐이고, 오케스트레이션은 **Runner 루프**가 맡는다. OpenAI 모델에 최적화돼 있지만 LiteLLM 연동으로 타사 모델도 쓸 수 있다. MCP 도구 연결을 네이티브 지원한다.

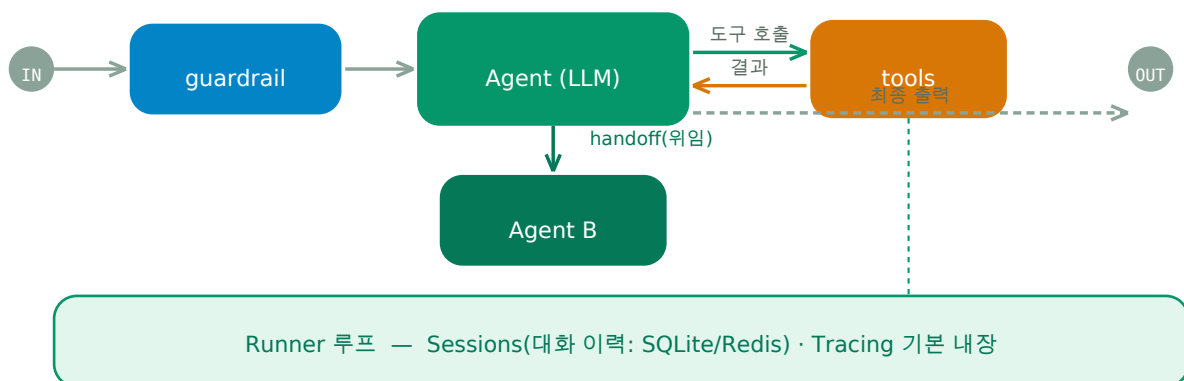
상황	권장
GPT 중심 워크플로를 빠르게 출시	Agents SDK (가장 적은 코드)
에이전트 분기·위임이 단순(위임 체인 수준)	Agents SDK handoffs
입출력 안전성 검증이 중요	Agents SDK guardrails
복잡한 그래프형 상태 머신·세밀한 롤백	LangGraph 고려
역할 분담형 다중 에이전트 협업	CrewAI 고려

한 줄 정의

Agents SDK = "다섯 개의 프리미티브(Agent·Tool·Handoff·Guardrail·Session)를 Runner 루프가 돌려주는 경량 에이전트 SDK".

2 핵심 멘탈 모델: Agent + Runner 루프

- **Agent** — instructions(시스템 프롬프트)·tools·handoffs·output_type을 가진 LLM 구성체.
- **Runner** — 모델 호출 → 도구 실행 → 결과 재투입을 **최종 출력이 나올 때까지 반복**하는 실행 루프.
- **Handoff** — 대화 제어권을 다른 전문 에이전트로 이양(단방향).
- **Guardrail** — 입력/출력을 병렬 검증, 위반 시 tripwire로 실행 중단.
- **Session** — 실행 간 대화 이력을 자동 저장·주입.



▲ Runner 루프: guardrail 통과 → Agent가 도구 호출/결과 관찰을 반복 → 최종 출력. 필요 시 handoff로 다른 에이전트에 위임.

LangGraph가 그래프(노드·엣지)를 명시적으로 그린다면, Agents SDK는 루프 + 위임이라는 단순한 모델을 쓴다. 그래서 코드가 짧고 학습이 빠르지만, 임의 토폴로지의 정밀 제어는 그래프 프레임워크가 유리하다.

3 Agent — instructions·tools·구조화 출력

Agent는 name, instructions, tools, handoffs, output_type으로 구성된다. output_type에 Pydantic 모델을 주면 구조화 출력이 강제되고, 이것이 생성되는 순간 루프가 종료된다. instructions는 함수로 줘서 런타임 동적 생성도 가능하다.

```
from agents import Agent, Runner
from pydantic import BaseModel

class Answer(BaseModel):
    summary: str
    confidence: float

agent = Agent(
    name="Researcher",
    instructions="You are a precise research assistant.",
    model="gpt-5.2",
    tools=[web_search],
    output_type=Answer,      # structured output ends the loop
)
result = Runner.run_sync(agent, "Summarize LangGraph in one line")
print(result.final_output)  # -> Answer(...)
```

4 Tools — function tool과 호스티드 도구

@function_tool을 붙이면 파이썬 함수의 시그니처·독스트링에서 스키마를 자동 생성한다. 그 외 OpenAI가 호스팅하는 내장 도구(웹 검색, 파일 검색, 코드 인터프리터, 컴퓨터 사용)와 에이전트를 도구로 노출하는 agent.as_tool(), MCP 서버 도구를 지원한다.

```
from agents import function_tool, WebSearchTool

@function_tool
def get_weather(city: str) -> str:
    """Return current weather for the given city."""
    return f"{city}: 23C, clear"

agent = Agent(name="Helper",
    tools=[get_weather, WebSearchTool()], # custom + hosted
    instructions="Use tools when needed.")
```

- 호스티드 도구 — WebSearchTool, FileSearchTool, CodeInterpreterTool, ComputerTool 등(Responses API 기반).
- Agents as tools — agent.as_tool(...)로 다른 에이전트를 도구처럼 호출(중앙 오케스트레이션).
- MCP — MCPServerStdio 등으로 MCP 서버의 도구를 그대로 연결.

5 Runner — 실행 루프와 RunResult

Runner.run(agent, input)(async) / run_sync / run_streamed 세 진입점이 있다. 루프는 (1) 모델 호출 → (2) 도구 호출 있으면 실행 후 결과 재투입 → (3) handoff면 에이전트 교체 → (4) 최종 출력(output_type 일치 또는 도구 호출 없는 텍스트)이 나오면 종료. max_turns 초과 시

MaxTurnsExceeded 예외가 난다.

```
result = await Runner.run(agent, "hello", max_turns=10)
result.final_output      # final answer (str or output_type instance)
result.last_agent        # which agent produced the final answer
result.new_items         # messages/tool calls/handoffs from this run
result.to_input_list()   # full history -> feed into the next run
```

대화 이어가기 두 가지 방법

수동: `result.to_input_list()`에 새 사용자 메시지를 붙여 다음 `Runner.run`에 전달. 자동: `Session`을 쓰면 SDK가 이력 저장·주입을 대신한다(8장).

6 Handoffs — 에이전트 간 위임

`handoffs=[agent_b, ...]`를 주면 모델에게 위임용 도구가 노출되고, 모델이 호출하면 **대화 제어권이 통째로 넘어간다**(단방향). 기본적으로 대상 에이전트가 **전체 대화 이력**을 받으며, `handoff(agent, input_filter=...)`로 전달 범위·콜백을 커스터마이징한다.

```
from agents import Agent, handoff

triage = Agent(
    name="Triage",
    instructions="Route billing questions to Billing, bugs to Support.",
    handoffs=[billing_agent,
               handoff(support_agent, input_filter=strip_old_messages)],
)
# result.last_agent tells you who finished the conversation
```

7 Guardrails — 입출력 검증과 tripwire

가드레일은 본 에이전트와 **병렬로** 빠른 모델/함수를 돌려 입력(첫 에이전트의 `input`) 또는 출력(마지막 에이전트의 `output`)을 검증한다. `GuardrailFunctionOutput(tripwire_triggered=True)`를 반환하면 **즉시 실행이 중단**되고 `InputGuardrailTripwireTriggered` 예외가 발생한다 — 비싼 모델 호출 전에 오남용을 차단하는 비용 절감 장치이기도 하다.

```
from agents import input_guardrail, GuardrailFunctionOutput

@input_guardrail
async def block_homework(ctx, agent, user_input):
    bad = "do my homework" in str(user_input).lower()
    return GuardrailFunctionOutput(output_info=None,
                                    tripwire_triggered=bad)

agent = Agent(name="Tutor", instructions="...",
               input_guardrails=[block_homework])
```

8 Sessions — 대화 이력 관리

기본적으로 `Runner.run()`은 매번 새 대화다. `session=...`을 넘기면 SDK가 **이전 이력 주입 + 새 항목 저장**을 자동으로 처리한다. 단일 프로세스는 `SQLiteSession`, 스케일 아웃 서버는 `RedisSession`, 분기 대화는 `AdvancedSQLiteSession`을 쓴다.

```
from agents import SQLiteSession
```

```

session = SQLiteSession("user-42", "chats.db")
await Runner.run(agent, "My name is Taehun", session=session)
r = await Runner.run(agent, "What is my name?", session=session)
# -> remembers prior turns automatically

```

9 Context — 의존성 주입

`Runner.run(..., context=obj)`로 임의 파이썬 객체를 넘기면 모든 도구·가드레일·핸드오프 콜백이 `RunContextWrapper[T]`로 받는다. LLM에게는 전송되지 않는 로컬 상태(DB 커넥션, 사용자 정보, 로거)를 타입 안전하게 공유하는 DI 메커니즘이다. LLM에게 보일 데이터는 instructions/입력/도구 반환값으로 넣어야 한다.

```

from dataclasses import dataclass
from agents import RunContextWrapper, function_tool

@dataclass
class Daps:
    user_id: str
    db: object

@function_tool
def my_orders(ctx: RunContextWrapper[Daps]) -> str:
    return ctx.context.db.orders_for(ctx.context.user_id)

await Runner.run(agent, "show my orders", context=Daps("u42", db))

```

10 Streaming

`Runner.run_streamed()`가 `RunResultStreaming`을 반환하고, `stream_events()` 비동기 이터레이터로 이벤트를 받는다. 토큰 단위 델타는 `raw_response_event`, 도구 호출/메시지 완료 같은 상위 이벤트는 `run_item_stream_event`로 온다.

```

result = Runner.run_streamed(agent, "tell me a story")
async for ev in result.stream_events():
    if ev.type == "raw_response_event":
        ... # token deltas for chat UI
    elif ev.type == "run_item_stream_event":
        ... # tool called / message completed etc.

```

11 Tracing — 기본 내장 관측성

- **기본 활성화** — LLM 호출·도구·핸드오프·가드레일이 trace/span 계층으로 자동 기록, OpenAI 대시보드에서 조회.
- `trace('workflow')` 컨텍스트로 여러 run을 하나의 워크플로 트레이스로 묶음.
- **커스텀 프로세서** — Logfire·AgentOps·Langfuse 등 외부 관측 스택으로 전송 가능.
- 민감 데이터는 `RunConfig(trace_include_sensitive_data=False)`로 제외, 환경변수로 전체 비활성화 가능.

12 멀티 에이전트 패턴

패턴	구조	적합
Handoffs(분산)	에이전트가 대화 제어권을 통째로 이양	트리아지→전문가 라우팅
Agents as tools(중앙)	오케스트레이터가 하위 에이전트를 도구로 호출	제어권 유지·결과만 수집
LLM 판단 오케스트레이션	모델이 도구/위임을 스스로 선택	개방형 작업
코드 오케스트레이션	파이썬 제어문으로 run을 체이닝	결정적 파이프라인

핵심 선택지는 **제어권을 넘길 것인가(handoff)**, **잡아둘 것인가(as_tool)**이다. 대화형 라우팅(고객지원 트리아지)은 handoff가, 결과만 모아 합성하는 작업은 as_tool이 자연스럽다.

13 Agents SDK vs LangGraph 비교

항목	OpenAI Agents SDK	LangGraph
추상화	Agent + Runner 루프(경량 SDK)	상태 그래프 노드/엣지(저수준)
핵심 모델	Agents·Tools·Handoffs·Guardrails·Sessions	State·Node·Edge + 체크포인트
제어 흐름	루프 + 위임(암묵적)	그래프로 명시적 분기·순환
학습 곡선	가장 완만	가파름
상태/영속	Sessions(대화 이력 중심)	체크포인트(임의 State 스냅샷)
휴먼인터럽트	승인 플로우(도구 승인)	interrupt() / resume(임의 지점)
안전장치	guardrails + tripwire 내장	가드레일은 직접 구성
관측성	Tracing 기본 내장(OpenAI 대시보드)	LangSmith 연동
모델	OpenAI 최적화(+LiteLLM로 확장)	프로바이더 중립
대표 선택	GPT 중심·빠른 출시·단순 위임	복잡 상태 머신·멀티 프로바이더·정밀 제어

현장 패턴: 첫 버전은 Agents SDK로 출시하고, 그래프형 문제(복잡 순환·임의 지점 재개·세밀한 롤백)에 부딪히면 LangGraph로 옮기는 팀이 많다. 반대로 OpenAI 종속이 부담이면 처음부터 프로바이더 중립 스택을 고른다.

14 자주 하는 실수 모음

증상	원인	해결
대화가 매번 초기화됨	session 미전달	SQLiteSession 등 session= 전달
MaxTurnsExceeded 예외	도구 루프 폭주/턴 제한 초과	max_turns 조정 + 도구·지시 점검
컨텍스트를 모델이 모름	context는 LLM에 안 감	프롬프트/도구 반환값으로 노출
guardrail이 안 도는 느낌	입력 가드레일은 첫 에이전트만	마지막 출력은 output_guardrail로
handoff 후 이력이 너무 큼	전체 대화가 기본 전달	handoff(input_filter=...)로 축소
구조화 출력이 안 끝남	output_type 미지정	output_type=PydanticModel 지정
타사 모델 사용 불가 오해	기본은 OpenAI	LiteLLM 연동으로 멀티 프로바이더

15 API 빠른 레퍼런스

목적	코드
에이전트	<code>Agent(name, instructions, tools, handoffs, output_type)</code>
커스텀 도구	<code>@function_tool def f(...) -> str</code>
호스팅 도구	<code>WebSearchTool()</code> / <code>FileSearchTool()</code> / <code>CodeInterpreterTool()</code>
실행	<code>await Runner.run(agent, input)</code> / <code>run_sync</code> / <code>run_streamed</code>
결과	<code>result.final_output</code> / <code>last_agent</code> / <code>to_input_list()</code>
위임	<code>handoffs=[b]</code> / <code>handoff(b, input_filter=...)</code>
가드레일	<code>@input_guardrail</code> / <code>@output_guardrail + tripwire</code>
세션	<code>SQLiteSession("id") -> Runner.run(..., session=s)</code>
컨텍스트	<code>Runner.run(..., context=obj) -> RunContextWrapper</code>
트레이싱	<code>with trace("workflow"): ...</code>

Agents SDK는 빠르게 진화합니다. 본 문서는 2026-06 시점 기준이며, 정확한 시그니처는 openai.github.io/openai-agents-python과 설치된 버전 릴리스 노트로 최종 확인하세요.

Q

지식 점검 퀴즈 (객관식 20문항)

정답·해설은 다음 장에 있습니다. 보기 중 하나를 고르세요.

Q1. Agents SDK의 핵심 프리미티브 조합으로 가장 정확한 것은?

- A. State·Node·Edge
- B. Agents·Tools·Handoffs·Guardrails·Sessions
- C. Crews·Tasks·Process
- D. Chain·Prompt·Parser

Q2. Agents SDK의 전신이 된 실험 프로젝트는?

- A. AutoGPT
- B. Swarm
- C. BabyAGI
- D. LangServe

Q3. Runner 루프가 종료되는 조건은?

- A. 항상 1턴 후
- B. 최종 출력(output_type 일치 또는 도구 호출 없는 텍스트) 생성 시
- C. 도구 호출 발생 시
- D. 세션 저장 시

Q4. 구조화 출력을 강제하는 Agent 설정은?

- A. output_type=PydanticModel
- B. instructions
- C. max_turns
- D. as_tool()

Q5. @function_tool의 핵심 동작은?

- A. 함수를 비동기로 변환
- B. 시그니처·독스트링에서 도구 스키마 자동 생성
- C. 함수를 에이전트로 변환
- D. 토큰 캐싱

Q6. 호스팅드 도구가 아닌 것은?

- A. WebSearchTool
- B. FileSearchTool
- C. CodeInterpreterTool
- D. SQLiteSession

Q7. handoff의 기본 동작으로 옳은 것은?

- A. 결과만 반환하고 제어권 유지
- B. 대상 에이전트가 전체 대화를 받고 제어권이 넘어감(단방향)
- C. 양방향 핑퐁
- D. 이력 없이 빈 컨텍스트 전달

- Q8. 전달 이력을 줄이며 위임하려면?
- A. `handoff(agent, input_filter=...)`
 - B. `max_turns=1`
 - C. session 삭제
 - D. `output_type` 지정
- Q9. 가드레일이 위반을 감지하면 일어나는 일은?
- A. 경고 로그만 남김
 - B. tripwire가 걸려 실행이 중단되고 예외 발생
 - C. 자동 재시도
 - D. 모델 교체
- Q10. 입력 가드레일이 실행되는 대상은?
- A. 모든 에이전트
 - B. 첫 번째 에이전트
 - C. 마지막 에이전트
 - D. 도구만
- Q11. 실행 간 대화 이력을 자동 관리하는 메커니즘은?
- A. Checkpointer
 - B. Session(SQLiteSession/RedisSession)
 - C. VectorStore
 - D. `trace()`
- Q12. 수동으로 다음 턴에 이력을 넘기는 방법은?
- A. `result.to_input_list()` + 새 메시지
 - B. `result.raw`
 - C. `agent.clone()`
 - D. `ctx.context`
- Q13. Runner.run의 context 인자에 대해 옳은 것은?
- A. LLM 프롬프트에 자동 포함된다
 - B. 도구·콜백에 주입되는 로컬 상태이며 LLM에는 전송되지 않는다
 - C. 토큰 비용을 줄인다
 - D. 세션을 대체한다
- Q14. 토큰 단위 스트리밍 델타를 받는 이벤트 타입은?
- A. `run_item_stream_event`
 - B. `raw_response_event`
 - C. `tool_event`
 - D. `handoff_event`
- Q15. 트레이싱에 대한 설명으로 옳은 것은?
- A. 기본 비활성화
 - B. 기본 활성화이며 외부 프로세서로도 전송 가능
 - C. 유료 전용
 - D. 도구 호출은 기록 안 됨

Q16. 중앙 오케스트레이터가 제어권을 유지한 채 하위 에이전트를 쓰는 방법은?

- A. handoffs
- B. agent.as_tool()
- C. tripwire
- D. RedisSession

Q17. 루프 폭주를 막는 안전장치는?

- A. max_turns + MaxTurnsExceeded
- B. output_json
- C. @persist
- D. interrupt()

Q18. 타사(비 OpenAI) 모델을 쓰는 대표 경로는?

- A. 불가능
- B. LiteLLM 연동(LiteLLMModel)
- C. YAML 설정
- D. handoff

Q19. Agents SDK vs LangGraph 비교로 가장 정확한 것은?

- A. SDK가 더 저수준이다
- B. SDK=경량 루프+위임으로 빠른 출시, LangGraph=명시적 그래프로 정밀 제어
- C. LangGraph는 연속성이 없다
- D. 둘 다 OpenAI 전용

Q20. result.last_agent가 알려주는 것은?

- A. 최초 호출 에이전트
- B. 최종 답을 만든 에이전트(핸드오프 추적)
- C. 가장 비싼 에이전트
- D. 도구 개수

A

정답 및 해설

- Q1** 정답: B
핵심 프리미티브는 Agents·Tools·Handoffs·Guardrails·Sessions 다섯 개. 실행은 Runner, 관측은 내장 Tracing.
- Q2** 정답: B
실험작 Swarm의 프로덕션 지향 후속이 Agents SDK다.
- Q3** 정답: B
루프는 도구 실행·핸드오프를 반복하다 최종 출력(output_type 일치 or 도구 호출 없는 텍스트)이 나오면 종료.
- Q4** 정답: A
output_type에 Pydantic 모델을 주면 구조화 출력이 강제되고 생성 시 루프가 끝난다.
- Q5** 정답: B
@function_tool은 함수 시그니처와 독스트링으로 도구 이름·설명·파라미터 스키마를 자동 생성.
- Q6** 정답: D
SQLiteSession은 세션(대화 이력) 저장소다. 나머지는 OpenAI 호스팅 도구.
- Q7** 정답: B
handoff는 단방향 제어권 이양이며 기본적으로 전체 대화 이력이 대상 에이전트로 전달된다.
- Q8** 정답: A
handoff(agent, input_filter=...)로 전달 이력을 가공/축소할 수 있다.
- Q9** 정답: B
tripwire_triggered=True면 Runner가 즉시 중단하고 *GuardrailTripwireTriggered 예외를 던진다.
- Q10** 정답: B
입력 가드레일은 첫 에이전트에서, 출력 가드레일은 마지막 에이전트에서 실행된다.
- Q11** 정답: B
Session(SQLite/Redis/AdvancedSQLite)이 이력 주입과 저장을 자동 처리한다.
- Q12** 정답: A
result.to_input_list()에 새 사용자 메시지를 이어붙여 다음 Runner.run에 전달한다.
- Q13** 정답: B
context는 도구·가드레일·콜백에 주입되는 로컬 DI 객체이며 LLM에는 전송되지 않는다.
- Q14** 정답: B
raw_response_event가 토큰 델타, run_item_stream_event가 도구 호출/메시지 완료 등 상위 이벤트.
- Q15** 정답: B
트레이싱은 기본 활성화이고 OpenAI 대시보드 외에 커스텀 프로세서로 외부 스택 전송이 가능하다.
- Q16** 정답: B
as_tool()은 하위 에이전트를 도구로 노출해 오케스트레이터가 제어권을 유지한다. handoff는 제어권 이양.
- Q17** 정답: A
max_turns 초과 시 MaxTurnsExceeded 예외로 루프 폭주를 차단한다.

Q18 정답: B

LiteLLM 연동(LitellmModel)으로 다양한 프로바이더 모델을 쓸 수 있다.

Q19 정답: B

SDK는 경량 루프+위임으로 빠른 출시에, LangGraph는 명시적 그래프·체크포인트로 정밀 제어·복잡 상태에 강하다.

Q20 정답: B

last_agent는 최종 답을 생성한 에이전트를 가리켜 핸드오프 체인을 추적하게 해준다.